

// DOCUMENTAÇÃO TÉCNICA • HANDOFF

Projeto Jarvis

Board de tarefas (matriz de prioridades) para gestão de PCM — arquitetura, decisões de design, fluxo de publicação e pendências, para continuar em uma instância do Claude Code.

REPOSITÓRIO	<code>github.com/hayralde/Jarvis</code>
HOSPEDAGEM	GitHub Pages (site estático)
PASTA LOCAL	<code>C:\Users\Jose.cordeiro\OneDrive - RRP\Documentos\GitHub\Jarvis</code>
AUTOR / DONO	Jose Cordeiro (hayralde)
GERADO POR	Claude, em sessão Cowork
DATA	1 de setembro de 2026



01 Visão geral

Jarvis é um board pessoal de tarefas para organização de trabalho de PCM (Planejamento e Controle de Manutenção), estruturado como **Matriz de Eisenhower** — as quatro categorias clássicas de urgência × importância. É uma página web publicada no GitHub Pages, sem servidor e sem banco de dados: todo o estado (quais tarefas existem, em qual quadrante, o que já foi concluído) vive dentro do HTML e do `localStorage` do navegador de quem abre a página.

O fluxo de trabalho pensado é o seguinte: o usuário manda tarefas para o Claude pelo chat (texto, PDF, áudio transcrito), o Claude registra essas tarefas e publica a atualização no site. Uma vez publicado, o usuário (ou qualquer pessoa com o link) pode abrir a página e interagir diretamente nela — mover tarefas entre quadrantes, marcar como concluída, ou adicionar uma nova tarefa ali mesmo.

CONTEXTO DO USUÁRIO

Jose trabalha na área de PCM (planejamento e controle de manutenção industrial). O board reflete tarefas reais de trabalho: aprovações de pedido de compra, requisições, e pendências com fornecedores/prestadoras de serviço.

Por que a Matriz de Eisenhower

Foi a interpretação adotada quando o usuário pediu uma "matriz" para organizar tarefas (a fala original ficou incompleta/cortada). Os quatro quadrantes usados:

QUADRANTE	CRITÉRIO	AÇÃO
Q1	Urgente + importante	Fazer agora
Q2	Importante, não urgente	Planejar
Q3	Urgente, baixo impacto	Delegar
Q4	Baixo valor	Eliminar

Se essa não for a matriz que o usuário tinha em mente originalmente, vale confirmar com ele antes de continuar evoluindo o conceito.

02 Arquitetura e publicação

Onde tudo mora

- **Repositório GitHub:** `github.com/hayralde/Jarvis` — repositório público, um único arquivo: `index.html`, servido via GitHub Pages (branch `main`).
- **Clone local:** feito via GitHub Desktop, na máquina Windows do usuário, em `C:\Users\Jose.cordeiro\OneDrive - RRP\Documentos\GitHub\Jarvis`. Está dentro de uma pasta sincronizada pelo OneDrive.
- **Autenticação:** feita pelo GitHub Desktop (login OAuth do usuário), não por token pessoal manuseado pelo Claude.

Como a publicação funciona hoje (ambiente Cowork)

Este projeto foi desenvolvido numa sessão do **Cowork** (Claude rodando num container na nuvem, sem terminal/git nativo com acesso à internet do GitHub, e sem poder manusear tokens diretamente em comandos — bloqueado por um classificador de segurança do ambiente). Para contornar isso, o fluxo de publicação usado foi:

1. Claude escreve o novo `index.html` localmente no container.
2. Claude transfere esse arquivo para a pasta clonada no computador do usuário, via ponte de arquivos (o computador precisa estar com o Claude Desktop aberto e vinculado à sessão).
3. Claude usa **controle de computador** (computer use) para abrir o GitHub Desktop, clicar em "Commit to main" e depois em "Push origin" — literalmente clicando nos botões da interface, com aprovação prévia do usuário para controlar aquele aplicativo.

LIMITAÇÕES CONHECIDAS DESSE FLUXO

- Push direto via `git` + token pessoal (PAT) foi tentado primeiro e **bloqueado** pelo classificador de segurança do Cowork (não permite embutir credenciais em comandos de shell).
- O bridge com o computador do usuário cai com frequência ("device not connected" / "computer use not available") — nesses casos o arquivo às vezes é escrito mas o commit/push manual fica pendente até a conexão voltar.
- O GitHub Desktop, rodando numa pasta sincronizada pelo OneDrive, às vezes demora alguns segundos para detectar a mudança no arquivo (o watcher de filesystem não dispara na hora) — geralmente resolve trocando de aba (Changes → History → Changes) para forçar um refresh.
- Controle de computador não tem modo "segundo plano" disponível nessa máquina — a janela do GitHub Desktop vem pra frente da tela por alguns segundos a cada publicação.

COMO ISSO MUDA NUMA INSTÂNCIA DO CLAUDE CODE

Claude Code roda com acesso real a terminal e sistema de arquivos (local ou num container próprio, dependendo de como for configurado), então esse fluxo de automação de tela deixa de ser necessário. Recomendado:

- Configurar `gh auth login` (GitHub CLI) ou uma chave SSH no ambiente onde o Claude Code roda, para autenticação nativa de git.
- Com isso, Claude Code pode rodar `git add / commit / push` diretamente pelo terminal, sem precisar de GitHub Desktop nem de controle de tela.
- Alternativa mais robusta ainda: configurar uma **GitHub Action** que faz deploy automático do `index.html` para o GitHub Pages a cada push — remove qualquer necessidade de clique manual, em qualquer ambiente.

03 Design e identidade visual

A primeira versão foi um dashboard escuro genérico (gradiente radial, cards com glassmorphism) — rejeitada pelo usuário por parecer "template de IA". A segunda versão adotou uma identidade de **console/HMI industrial** (a mesma família de interface usada em salas de controle de planta): tipografia monoespaçada, LEDs de status com cores reais de gestão de alarme, grid técnico de fundo. Essa direção foi mantida; só a paleta clara/escura mudou depois (o usuário achou o fundo escuro desagradável).

Paleta atual (tema claro)

TOKEN	HEX	USO
--bg	#f3f4ee	Fundo da página
--panel	#fbfcf8	Fundo dos cards de quadrante
--text	#232922	Texto principal
--red	#c73321	Q1 — crítico / alta prioridade
--green	#188a4c	Q2 — rotina / normal
--amber	#a86a05	Q3 — atenção
--grey	#727b6b	Q4 — baixo valor

Tipografia

IBM Plex Mono (títulos, labels, dados, botões — identidade técnica) + **IBM Plex Sans** (corpo de texto das tarefas — legibilidade), carregadas via Google Fonts. Ambas com pesos 400/500/600/700.

Elemento de assinatura

A barra de status no topo (estilo terminal: "JARVIS" + cursor piscando + LED "online") e os cantos em formato de mira/reticle nos quadrantes ao passar o mouse — reforçam a metáfora de painel de controle sem depender de decoração genérica.

04 Modelo de dados e persistência

Todo o estado das tarefas vive em três chaves do `localStorage` do navegador — não existe backend, banco de dados, nem sincronização entre dispositivos.

CHAVE	CONTEÚDO
<code>jarvis_tasks_v1</code>	Array JSON com as tarefas atualmente no board (id, quadrante, título, meta, data)
<code>jarvis_seen_ids_v1</code>	Array de ids de tarefas "semente" já processadas nesse navegador (ver mecanismo de merge abaixo)
<code>jarvis_completed_total</code>	Contador de tarefas concluídas (exibido na barra de status)

O problema que o mecanismo de merge resolve

Como o estado vive no navegador do usuário, simplesmente editar a lista de tarefas no código-fonte do `index.html` não faz as tarefas novas aparecerem para quem já abriu a página antes (o navegador já tem seu próprio estado salvo, que tem prioridade). A solução foi um array `SEED_TASKS` no JavaScript — a "lista mestra" que o Claude edita a cada nova tarefa recebida no chat — combinado com uma lógica de merge que roda a cada carregamento de página:

```
function loadAndMergeTasks(){
  var tasks = loadRaw(STORAGE_KEY) || [];
  var seen = loadRaw(SEEN_KEY) || [];
  var changed = false;
  SEED_TASKS.forEach(function(seed){
    var alreadySeen = seen.indexOf(seed.id) !== -1;
    if(alreadySeen) return; // já processado antes (pode já
                          // ter sido movido/concluído)

    seen.push(seed.id);
    changed = true;
    var alreadyInTasks = tasks.some(function(t){
      return t.id === seed.id;
    });
    if(!alreadyInTasks){ tasks.push(Object.assign({}, seed)); }
  });
  if(changed){ saveTasks(tasks); saveSeen(seen); }
  return tasks;
}
```

Regra prática: **cada tarefa em `SEED_TASKS` precisa de um id estável e único** (nunca reaproveitar nem mudar o id de uma tarefa já publicada). Ids atuais em uso: `t1`, `t2`, `seed-bruno-engates`, `seed-rc-rogerio`.

LIMITAÇÃO IMPORTANTE PARA O PRÓXIMO PASSO

Esse mecanismo resolve bem *adicionar* tarefas novas, mas não resolve *editar o texto* de uma tarefa já mesclada no navegador de alguém (o merge só olha se o id já existe, não compara conteúdo). Se isso

virar um problema recorrente, vale considerar migrar para um backend real (ver seção 6) em vez de continuar empilhando lógica de merge no localStorage.

Testes realizados

Antes de cada publicação, o comportamento foi validado com Playwright (navegador headless) cobrindo três cenários: navegador novo (sem dado nenhum), navegador com dado do formato antigo (migração), e navegador onde uma tarefa semente já tinha sido concluída (não pode reaparecer). Os três passaram. Recomenda-se manter esse hábito de teste antes de publicar mudanças na lógica de dados.

05 Funcionalidades interativas

AÇÃO	COMO FUNCIONA
Mover tarefa	Botões numerados (1–4) em cada card, ou arrastar-e-soltar (drag and drop) entre quadrantes
Concluir tarefa	Botão "✓ concluir" — remove o card com animação e soma no contador da barra de status
Adicionar tarefa	" + nova tarefa " no rodapé de cada quadrante — abre um campo de texto inline

Todas as três ações persistem imediatamente no `localStorage` — sobrevivem a recarregar a página, mas não saem daquele navegador específico.

06 O que falta / ideias para evoluir

- **Sincronização entre dispositivos:** hoje, tarefas concluídas/movidas no celular não aparecem no computador e vice-versa, porque cada um tem seu próprio `localStorage`. Se isso for um requisito real, a página precisa de um backend (ex.: Supabase, Firebase, ou até salvar num arquivo JSON no próprio repo via GitHub API a cada mudança).
- **Deploy automático:** configurar GitHub Actions para publicar a cada push, eliminando dependência de GitHub Desktop / controle de tela.
- **Automação do WhatsApp Desktop:** usuário quer testar se o Claude consegue operar o WhatsApp instalado na máquina (mesmo mecanismo de controle de computador usado para o GitHub Desktop) para enviar mensagens — por exemplo, mandar os números de RC direto pro Rogério. Ainda não testado.
- **Assistente de lembrete:** o usuário quer que o Claude funcione como um assistente que ajuda a lembrar e executar tarefas do dia a dia (não só manter o board atualizado).

07 Pessoas e contexto de trabalho

NOME	PAPEL
Rogério	Coordenador do usuário
Bruno	Almoxarifado (transcrição de áudio original ficou "xerifado" — não confirmado)
Josi	Aprova pedidos de compra (mencionado na tarefa do Motor LX)
Thiago	Cria requisições / dá acesso para requisição

"Decorfios"/"Decorfius" (grafia inconsistente nas mensagens do usuário) é a prestadora de serviço responsável pelos motores mencionados nas tarefas.

08 Tarefas pendentes no momento deste documento

QUADRANTE	TAREFA	OBSERVAÇÃO
Q1	Falar com Bruno do almoxarifado: trocar os engates rápidos comprados errado e pedir os certos	Primeira coisa ao chegar na empresa
Q1	Buscar no sistema CS o número das 2 RC dos motores da Decorfios e enviar pro Rogério	Navegação dentro do sistema CS ainda não mapeada — perguntar ao usuário
Q1	Pedir pro Rogério falar pro Josi aprovar o pedido	Motor LX · Decorfius
Q2	Pedir acesso pro Thiago para ele criar requisição	—
—	Comprar unidade seladora	Bloqueada: usuário ainda não informou em quais áreas vai instalar — não foi adicionada ao board por esse motivo

Checklist de retomada

- ☐ Confirmar com o usuário em quais áreas a unidade seladora será instalada, e então adicionar a tarefa ao `SEED_TASKS`
- ☐ Confirmar se "Bruno do xerifado" é de fato "Bruno do almoxarifado"
- ☐ Mapear com o usuário como navegar no sistema "CS" até a tela/relatório das RCs
- ☐ Confirmar se "RC" significa Requisição de Compra (assumido, não confirmado explicitamente)
- ☐ Se for continuar em Claude Code: configurar autenticação git nativa (gh CLI ou SSH) para não depender mais de GitHub Desktop + controle de tela